PHẦN 1. MÔ TẢ BÀI TOÁN VÀ NGHIỆP VỤ

1.1. Bài toán

Hệ thống E-commerce tập trung vào chương trình Flash Sale với 2 vai trò người dùng chính:

1. SELLER tạo các campaign có khung giờ cố định, mỗi campaign gồm nhiều sản phẩm với giá bán và số lượng giới hạn.
2. CUSTOMER xem danh sách sản phẩm của campaign và đặt mua trong khung giờ diễn ra. Khi đặt mua, hệ thống kiểm tra tồn kho, tạo đơn hàng và gọi cổng thanh toán.

1.2. Vai trò người dùng

Vai trò	Quyền hạn	Cách có được
Khách	Xem danh sách sản phẩm của một campaign; Đăng ký; Đăng nhập.	Không cần tài khoản
CUSTOMER	Xem danh sách sản phẩm của campaign; Đặt mua sản phẩm trong khung giờ Flash Sale.	Tự đăng ký với role CUSTOMER
SELLER	Upload ảnh sản phẩm; Tạo campaign Flash Sale kèm danh sách sản phẩm.	Tự đăng ký với role SELLER

1.3. Các nghiệp vụ chính

1.3.1. Tài khoản và xác thực

- Đăng ký tài khoản với role SELLER hoặc CUSTOMER. Username và email phải là duy nhất; mật khẩu được lưu dưới dạng BCrypt hash.
- Đăng nhập bằng username và password, trả về access token (hiệu lực 5 phút) và refresh token (hiệu lực 7 ngày).
- Làm mới token bằng refresh token: cấp access token mới và refresh token mới giữ nguyên hạn cũ. Mỗi người dùng chỉ có một refresh token hợp lệ tại một thời điểm.

1.3.2. Flash Sale

- SELLER upload ảnh thumbnail (JPEG/PNG, tối đa 10 MB). Nội dung ảnh được lưu trên RustFS, metadata lưu trong MySQL.

- SELLER tạo campaign gồm tên, mô tả, thời gian bắt đầu/kết thúc (end_at phải sau start_at) và danh sách sản phẩm (tên, giá, tồn kho, thumbnail).
- Mọi người dùng có thể xem danh sách sản phẩm của một campaign kèm giá và số lượng sản phẩm còn lại.
- CUSTOMER đã đăng nhập đặt mua sản phẩm trong khung giờ campaign đang diễn ra. Hệ thống kiểm tra tồn kho, tính tổng tiền theo giá đã lưu, trừ kho, lưu đơn hàng và gọi cổng thanh toán; thanh toán thất bại thì toàn bộ thay đổi được rollback.

1.3.3. Khác

- Notification: khi tạo campaign, hệ thống đặt lịch một tác vụ chạy đúng thời điểm start_at để gửi email thông báo "Flash Sale is starting now!" tới toàn bộ người dùng. Việc gửi email hiện được giả lập bằng log.
- Tích hợp bên thứ 3: gọi trực tiếp API thanh toán (payment mock viết bằng Express) ngay trong luồng đặt hàng.

PHẦN 2. ĐẶC TẢ API

2.1. Quy ước chung

Base path: /api/v1. Các API không ghi "Public" yêu cầu header Authorization: Bearer <access_token>.

Định dạng: Tên trường JSON dùng snake_case; thời gian theo ISO 8601 (UTC).

2.1.1. Cấu trúc phản hồi (Response Envelope)

Mọi phản hồi (thành công hay lỗi) đều tuân theo cấu trúc JSON sau:

Trường	Ý nghĩa
status_code	Mã HTTP trạng thái phản hồi
message	Thông điệp mô tả kết quả
data	Dữ liệu trả về (chỉ có khi thành công; object, mảng hoặc null)
error	Tên lỗi theo HTTP reason phrase (chỉ xuất hiện khi có lỗi, ví dụ: 'Bad Request')
path	URL endpoint gây ra lỗi (chỉ xuất hiện khi có lỗi)
timestamp	Thời điểm xảy ra lỗi, ISO 8601 (chỉ xuất hiện khi có lỗi)

2.1.2. Mã lỗi

Mã	Ngữ cảnh xảy ra
400	Dữ liệu gửi lên không hợp lệ; mật khẩu nhập lại không khớp; hết hàng; ảnh không hợp lệ; end_at không sau start_at; campaign chưa/không còn diễn ra
401	Thiếu, sai hoặc hết hạn access token; sai thông tin đăng nhập; refresh token không hợp lệ
403	Đã đăng nhập nhưng sai vai trò, hoặc thiếu hồ sơ người bán
404	Không tìm thấy campaign, sản phẩm của campaign hoặc thumbnail
409	Username/email đã tồn tại, hoặc vi phạm ràng buộc dữ liệu
500	Lỗi lưu trữ hoặc lỗi hệ thống
502	Cổng thanh toán từ chối, timeout hoặc không kết nối được; đơn hàng bị rollback

2.2. Danh sách endpoint

#	Method	Path	Quyền
1	POST	/auth/register	Public
2	POST	/auth/login	Public
3	POST	/auth/refresh	Public (cần refresh token hợp lệ)
4	POST	/files/item-images	SELLER
5	POST	/flash-sales/campaign/create	SELLER
6	GET	/flash-sales/campaign/{campaignId}/items	Public
7	POST	/flash-sales/campaign/order	CUSTOMER

2.3. Chi tiết API

2.3.1. Đăng ký - POST /auth/register

Mô tả: Tạo tài khoản mới với vai trò SELLER hoặc CUSTOMER.

Yêu cầu:

Trường	Kiểu	Bắt buộc	Ghi chú
email	string	✓	Email hợp lệ, duy nhất
username	string	✓	Tối đa 50 ký tự, duy nhất
password	string	✓	Mật khẩu
retype_password	string	✓	Phải trùng password
role	string	✓	SELLER hoặc CUSTOMER
shop_name	string		Bắt buộc khi role = SELLER
phone	string		9-15 chữ số (CUSTOMER)
shipping_address	string		Địa chỉ giao hàng (CUSTOMER)

Phản hồi: 201, data = null, message: "User registered successfully".

2.3.2. Đăng nhập - POST /auth/login

Mô tả: Xác thực tài khoản bằng username và password, trả về access token và refresh token.

Yêu cầu:

Trường	Kiểu	Bắt buộc	Ghi chú
username	string	✓	Tên đăng nhập
password	string	✓	Mật khẩu

Phản hồi: 200, data bao gồm: access_token, refresh_token, token_type ('Bearer'), expires_at, username, role.

2.3.3. Làm mới token - POST /auth/refresh

Mô tả: Dùng refresh token còn hiệu lực để lấy access token mới; refresh token cũng được cấp lại và giữ nguyên hạn ban đầu.

Yêu cầu:

Trường	Kiểu	Bắt buộc	Ghi chú
refresh_token	string	✓	Refresh token đang lưu của người dùng

Phản hồi: 200, data bao gồm: access_token, refresh_token, token_type ('Bearer'), expires_at.

2.3.4. Upload ảnh sản phẩm - POST /files/item-images

Mô tả: Người bán upload ảnh thumbnail cho sản phẩm. Chỉ chấp nhận JPEG/PNG có nội dung đúng với content type khai báo; tối đa 10 MB mỗi file.

Yêu cầu (multipart/form-data):

Trường	Kiểu	Bắt buộc	Ghi chú
file	file	✓	Ảnh JPEG hoặc PNG

Phản hồi: 201, data bao gồm: id (dùng làm thumbnail_id), object_key.

2.3.5. Tạo campaign - POST /flash-sales/campaign/create

Mô tả: Người bán tạo campaign Flash Sale kèm danh sách sản phẩm. Hệ thống đặt lịch gửi thông báo tại start_at.

Yêu cầu:

Trường	Kiểu	Bắt buộc	Ghi chú
name	string	✓	Tên campaign
description	string		Mô tả
start_at	string	✓	Thời điểm bắt đầu, ISO 8601 UTC
end_at	string	✓	Thời điểm kết thúc, phải sau start_at
items	array	✓	Danh sách sản phẩm, không được rỗng
items[].name	string	✓	Tên sản phẩm
items[].price	number	✓	Giá bán > 0
items[].stock	integer	✓	Số lượng tồn kho > 0
items[].thumbnail_id	string		ID ảnh đã upload, phải thuộc người bán

Phản hồi: 201, data bao gồm: id, name, started_at, ended_at.

2.3.6. Xem sản phẩm của campaign - GET /flash-sales/campaign/{campaignId}/items

Mô tả: Lấy danh sách sản phẩm của một campaign cùng giá và số lượng tồn kho còn lại. Không cần đăng nhập.

Tham số đường dẫn:

Trường	Kiểu	Bắt buộc	Ghi chú
campaignId	string	✓	ID campaign

Phản hồi: 200, data là danh sách items: id, campaign_id, name, thumbnail_id, price, stock.

2.3.7. Đặt mua sản phẩm Flash Sale - POST /flash-sales/campaign/order

Mô tả: Khách hàng đặt mua một hoặc nhiều sản phẩm của campaign đang diễn ra. Trong cùng một transaction: kiểm tra và trừ tồn kho, tính tổng tiền, lưu đơn hàng PENDING, gọi cổng thanh toán; thanh toán thất bại thì rollback toàn bộ.

Yêu cầu:

Trường	Kiểu	Bắt buộc	Ghi chú

items	array	✓	Danh sách sản phẩm đặt mua, không được rỗng
items[].campaign_item_id	string	✓	ID sản phẩm của campaign
items[].quantity	integer	✓	Số lượng > 0

Phản hồi: 201, data bao gồm: id, total_amount, status (PENDING), items.

PHẦN 3. VẤN ĐỀ HIỆN TẠI CỦA HỆ THỐNG

1. **Tranh chấp tài nguyên & Bán hàng vượt tồn kho (Race condition)**

   FlashSaleServiceImpl.createOrder đọc tồn kho bằng findById, kiểm tra stock < quantity rồi gọi setStock(stock - quantity) trong một transaction. Hai khách cùng mua món cuối cùng tại một thời điểm: cả hai đọc thấy stock = 1, cả hai qua kiểm tra, cả hai đặt hàng thành công. Hệ thống bán nhiều hơn số hàng thực đang có.

2. **Nghẽn cổ chai do API bên thứ 3**

   - PaymentClientImpl.processPayment gọi cổng thanh toán bằng WebClient ở chế độ blocking với timeout 5 giây, chạy ngay trên thread xử lý request. Mọi đơn hàng đều phải chờ cổng thanh toán.
   - Nếu cổng chậm hoặc sập thì mọi đơn hàng chậm hoặc thất bại theo, và toàn bộ ứng dụng có thể ngừng phục vụ request.

3. **Quá tải hệ thống do Push Notification đồng loạt**

   Khi campaign bắt đầu, SchedulerServiceImpl findAll toàn bộ người dùng rồi gửi thông báo cho từng người trong một vòng lặp trên một thread. Ứng dụng cố gửi cho tất cả người dùng tại cùng một thời điểm; với lượng người dùng lớn, điều này làm quá tải cả ứng dụng lẫn nhà cung cấp email/push.

4. **Xung đột Scheduled Task khi scale hệ thống (Horizontal Scaling)**

   SchedulerConfig dùng SimpleAsyncTaskScheduler, lưu task trong bộ nhớ JVM, nên mỗi instance có một scheduler riêng. Task đặt lịch chỉ tồn tại trong bộ nhớ của một instance do đó:

   - Restart instance đó thì task biến mất.
   - Chạy nhiều instance thì task có thể chạy nhiều lần.

5. **Xử lý đồng bộ chậm**

   CreateOrder thực hiện mọi bước (kiểm tra tồn kho, ghi database, gọi thanh toán) ngay trong request HTTP và chỉ phản hồi khi tất cả đã xong. Khách hàng phải chờ bước chậm nhất; kể cả khi cổng thanh toán nhanh, thời gian phản hồi vẫn tăng theo mỗi bước được thêm vào, và không có bước nào có thể để chạy sau.

6. **Dual Write**

   Hai hệ thống được ghi trong một bước mà không đảm bảo tính atomic:

   - CreateOrder ghi vào MySQL rồi gọi dịch vụ thanh toán trong cùng một transaction.
   - FileServiceImpl.uploadItemImage ghi vào RustFS rồi mới ghi MySQL.

Do đó tiền có thể đã bị trừ trong khi đơn hàng bị rollback, hoặc file đã lưu trong khi bản ghi database thất bại. Hai bên rơi vào trạng thái lệch nhau.

7. **Tight Coupling**

   NotificationServiceImpl bị gắn cứng với một kênh gửi: inject trực tiếp EmailService và gọi thẳng nó. Muốn thêm kênh khác (ví dụ SmsService) phải sửa NotificationServiceImpl

8. **Chưa có Cache**

   Các truy vấn đọc database không được cache lại sau mỗi lần gọi. Trong lúc flash sale, hàng nghìn lượt đọc giống hệt nhau dội vào database cùng lúc.

9. **Chưa có Idempotency Key**

   Khi người dùng thấy màn hình xoay mãi, họ sẽ tưởng bị lag mà click lại nhiều lần từ đó sẽ tạo ra số lượng order không như mong muốn. Sử dụng Idempotency Key sẽ giúp ngăn chặn request bị trùng lặp

PHẦN 4. CƠ SỞ DỮ LIỆU



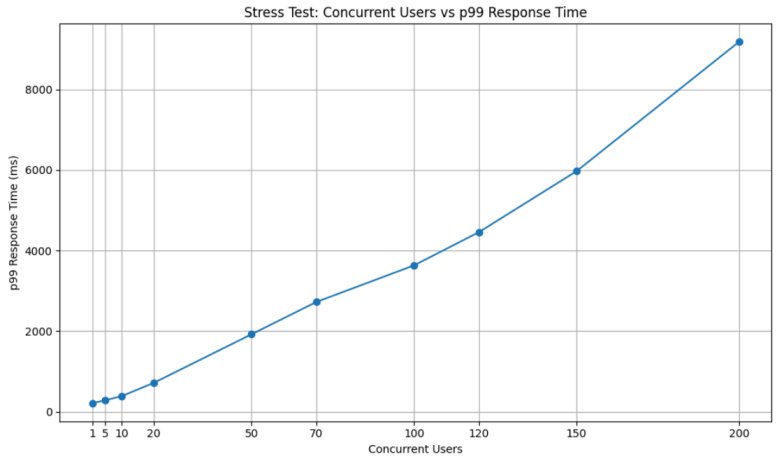
Phần 5 Thực nghiệm đo

Thực nghiệm đo:

1 Với stress test, khi nhiều người dùng đặt hàng 1 lúc.

Endpoint: /api/v1/flash-sales/campaign/order

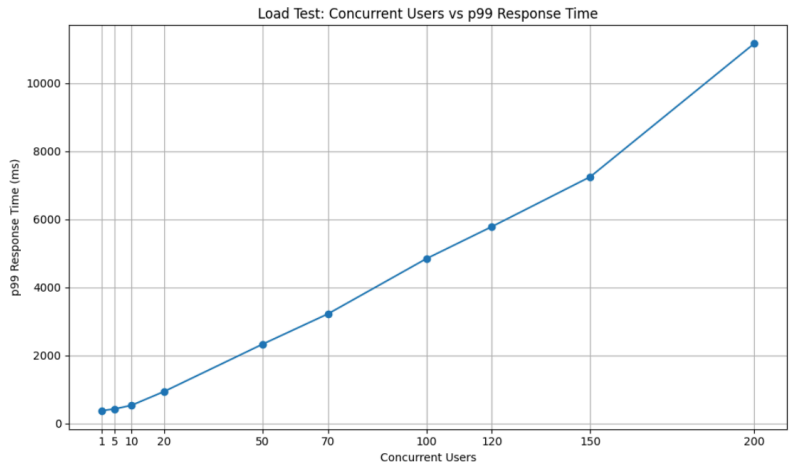
```
Final results:
         endpoint  concurrent_users  requests  successful  errors  non_200
0  campaign_order                 1      2000           0       0     2000
1  campaign_order                 5      2000           0       0     2000
2  campaign_order                10      2000           0       0     2000
3  campaign_order                20      2000           0       0     2000
4  campaign_order                50      2000           0       0     2000
5  campaign_order                70      2000           0       0     2000
6  campaign_order               100      2000           0       0     2000
7  campaign_order               120      2000           0      14     1986
8  campaign_order               150      2000           0       0     2000
9  campaign_order               200      2000           0      78     1922
```



Nhận xét: Khi tăng số lượng concurrent users, p99 response time tăng đáng kể. Hệ thống bắt đầu có dấu hiệu bão hòa từ khoảng 50 concurrent users, khi p99 vượt 1.9 giây. Từ 100 concurrent users trở lên, latency tăng mạnh và tại 120 concurrent users bắt đầu xuất hiện request errors. Ở 200 concurrent users, p99 đạt khoảng 9.18 giây và tỷ lệ lỗi đạt 3.9%, cho thấy hệ thống đã vượt quá mức tải mà hệ thống có thể xử lý ổn định.

2 Load test với api flash-sales/campaign/{campaignId}/items khi nhiều người cùng truy cập

```
Final results:
          endpoint  concurrent_users  requests  successful  errors  non_200  \
0  campaign_items                 1      2000        2000       0        0
1  campaign_items                 5      2000        2000       0        0
2  campaign_items                10      2000        2000       0        0
3  campaign_items                20      2000        2000       0        0
4  campaign_items                50      2000        2000       0        0
5  campaign_items                70      2000        2000       0        0
6  campaign_items               100      2000        2000       0        0
7  campaign_items               120      2000        2000       0        0
8  campaign_items               150      2000        2000       0        0
9  campaign_items               200      2000        2000       0        0
```



Nhận xét: hệ thống xử lý thành công toàn bộ 20,000 requests trong các mức tải từ 1 đến 200 concurrent users, không ghi nhận lỗi hoặc HTTP status khác 200. Tuy nhiên, response time tăng đáng kể khi concurrency tăng. p99 tăng từ 375.94 ms tại 1 concurrent user lên 11,161.32 ms tại 200 concurrent users. Throughput tăng nhanh ở mức tải thấp nhưng đạt trạng thái gần bão hòa từ khoảng 20 concurrent users, duy trì khoảng 26–28 requests/second khi concurrency tiếp tục tăng. Điều này cho thấy hệ thống vẫn đảm bảo tính đúng đắn và khả năng xử lý request nhưng bắt đầu gặp giới hạn về hiệu năng khi tải cao.